# Phase 1 Report

# Lexical Analyzer for Mini Java

CSCI415: Compiler Design-2026FALL-LECT-02


**Team members**

| | |
|---|---|
| Gehad Maher | 231000894 |
| Yassin Salah | 231000798 |
| Salma Ali | 231001369 |
| Assem Mostafa | 231000375 |
| Youssef Abdelrahman | 221000427 |

## 1. Introduction

We built a scanner for Mini Java that reads the source code, groups characters into tokens, skips whitespace and comments, builds a symbol table, and reports lexical errors with line and column numbers. The code is split into one file per job: `tokens.py`, `lexer.py`, `scan_ident.py`, `scan_symbols.py`, `scan_text.py`, `symbol_table.py`, `output.py` and `main.py`.

## 2. Tokens Implemented

| Category | Token names |
|---|---|
| Keywords | CLASS, PUBLIC, STATIC, VOID, INT, FLOAT, CHAR, IF, ELSE, WHILE, FOR, RETURN, BOOLEAN_TYPE, STRING_TYPE |
| Identifiers | IDENTIFIER |
| Literals | NUMBER, STRING, BOOLEAN |
| Operators | ASSIGN (=), MATHOP (+ - * / %), LOGOP (== != < > <= >= && \|\| !) |
| Delimiters | LPAR, RPAR, LCUR, RCUR, LSQU, RSQU, SEMICOLON, COMMA, DOT |

The language is case-sensitive: `class` is a keyword, `Class` is an identifier.

Whitespace and comments are skipped, and identifiers are saved in the symbol table.

## 3. How Errors Were Handled

Errors are printed with a message, line and column, in this format:

*Error: <message> at line X, column Y.*

The lexer does not stop at the first error. It reports the error, and keeps scanning so all errors in a file are found in one run.

**Invalid character** (@): reported by `lexer.py` when no scanner accepts the character.

**Invalid identifier** (1name): a word that starts with a digit but is not only digits.

**Unclosed string**: no closing quote before the end of the line; the opening quote's position is reported.

**Unclosed comment**: `/*` with no `*/`; the position where the comment started is reported.

## 4. Challenges Faced and How We Solved Them

**Working in parallel:** we agreed on a shared `tokens.py` and one function format for all scanners.

**`25` vs `1name`:** the lexer reads the whole word; all digits means a number, otherwise an invalid identifier error.

**`/` vs comments, `>=` vs `>`:** comments are checked before operators, and two-character operators are tried first.

**Line and column inside multi-line comments:** every scanner returns the new line and column.

**Unclosed comment hiding later tests:** we put that test at the end of the test file.

## 5. Testing

`test_input.txt` covers a valid program and each error type.

## Output screenshots:




